

HASHING

- **Hashing** is a technique to convert a range of key values into a range of indexes of an array.
- Hashing is the technique used for performing almost constant time search in case of insertion, deletion and find operation.
- **A hash table** is a data structure where we store a key value after applying the hash function. It is arranged in the form of an array i.e. addressed via a hash function. A hash table is divided into a number of buckets and each bucket is in turn capable of storing a numbers of records. We can say that a bucket has number of slots which is capable of holding one record. The array which supports hashing for storing records or searching records is called hash table.
- Thus, it becomes a data structure in which insertion and search operations are very fast irrespective of size of data. Hash Table uses array as a storage medium and uses hash technique to generate index where an element is to be inserted or to be located from.

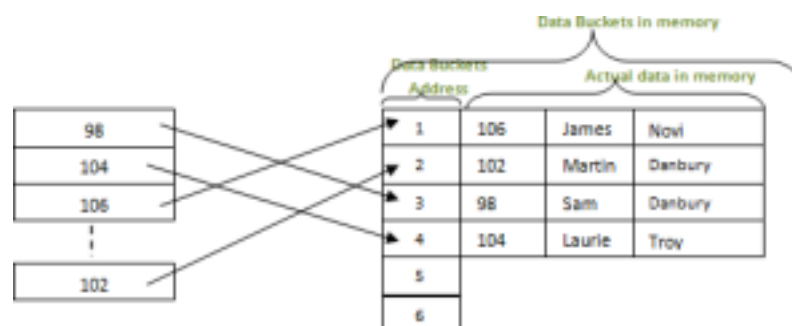
Types of Hashing

There are two types of Hashing:

- 1. Static hashing:** In static hashing, the hash function maps search-key values to a fixed set of locations.
- 2. Dynamic hashing:** In dynamic hashing a hash table can grow to handle more items. The associated hash function must change as the table grows.

Static Hashing

In this method of hashing, the resultant data bucket address will be always same. That means, if we want to generate address for EMP_ID = 103 using mod (5) hash function, it always result in the same bucket address 3. There will not be any changes to the bucket address here. Hence number of data buckets in the memory for this static hashing remains constant throughout. In our example, we will have five data buckets in the memory used to store the data.



Static hashing will be good for smaller databases where record size is previously known. If there is a growth in data, it results in serious problems like bucket overflow.

Unit-5: Hashing

Note:

- Key-value pairs are stored in a fixed size table called a *hash table*.
 - A hash table is partitioned into many **buckets**.
 - Each bucket has many **slots**.
 - Each slot holds one record.
 - A hash function $f(x)$ transforms the identifier (key) into an address in the hash table
- The **key density (or identifier density)** of a hash table is the ratio n/T • n is the number of keys in the table
 - T is the number of distinct possible keys
- The **loading density or loading factor** of a hash table is $a = n/(sb)$
 - s is the number of slots
 - b is the number of buckets

Dynamic Hashing

This hashing method is used to overcome the problems of static hashing – bucket overflow. In this method of hashing, data buckets grows or shrinks as the records increases or decreases. This method of hashing is also known as extendable hashing method. Let us see an example to understand this method.

Consider there are three records R1, R2 and R4 are in the table. These records generate addresses 100100, 010110 and 110110 respectively. This method of storing considers only part of this address

– especially only first one bit to store the data. So it tries to load three of them at address 0 and 1.



What will happen to R3 here? There is no bucket space for R3. The bucket has to grow dynamically to accommodate R3. So it changes the address have 2 bits rather than 1 bit, and then it updates the existing data to have 2 bit address. Then it tries to accommodate R3.



Now we can see that address of R1 and R2 are changed to reflect the new address and R3 is also inserted. As the size of the data increases, it tries to insert in the existing buckets. If no buckets are available, the number of bits is increased to consider larger address, and hence increasing the buckets. If we delete any record and if the data can be stored with lesser buckets, it shrinks the bucket size. It does the opposite of what we have seen above. This is how a dynamic hashing works.

Advantages of dynamic hashing

- Performance does not come down as the data grows in the system. It simply increases the memory size to accommodate the data.
- Since it grows and shrinks with the data, memory is well utilized. There will not be any unused memory lying.
- Good for dynamic databases where data grows and shrinks frequently

Hash Function

A function that transforms a key into a table index is called hash function. If h is a hash and k is key then $h(K)$ is called the hash function or hash of the key ' K '. It is the index in which the key k should be placed. Each key is mapped on a particular array index through hash function. If each key is mapped on a unique hash table then this situation is ideal but there may be possibility that hash function generating some hash table address for different keys & this situation is called collision.

Given a collection of items, a hash function that maps each item into a unique slot is referred to as a **perfect hash function**. If we know the items and the collection will never change, then it is possible to construct a perfect hash function.

A good hash function should:

- Minimize collisions.
- Be easy and quick to compute.
- Distribute key values evenly in the hash table.
- Use all the information provided in the key.

For missing collision we have to perform:

Choose a good hash table which performs minimum collision. Resolving the collision.

Technique for choosing hash function/Method of

hashing: Truncation method:-

In this method a part of key is considered as address, it can be some rightmost digit or leftmost digit.

Q. Apply the truncation method to get the hash index of table size of 100 for following

Keys: 82394561, 87139465, 83567271, 85943228

Solⁿ:

Table size = 100

Then, Take 2 rightmost digits for getting the hash table address.

Key value address

82394561 61

87139465 65

83567271 71

85943228 28

Example: $h(\text{key}) = a$

$h(823994561) = 61$

Modulus method/Division method:-

Modular method is best way for getting address from key. Take the key, do the modulus operation & get the remainder as address for hash table in order to minimize the Collision table size should a prime number consider keys.

Choose a number 'm (nearest prime number of the table size)'. Divide key to be mapped by the number m to calculate the remainder. Generally, the prime number is chosen to reduce the number of collisions.

$$H(k) = k \bmod m$$

Mid square method:-

In this method we square the key, after getting number we take some middle of that number as an address, suppose keys are 4 digits and maximum address = 100.

Key value square address $H(k)=L$

$$1456 \ 02119936 \ 19 \ h(1456) = 19$$

$$1892 \ 03579664 \ 79 \ h(1892) = 79$$

Here, The key is squared then the hash function is defined by $H(k)=L$, where L is obtained by deleting the digits from both the ends of key square.

Folding Method:

A key is partition into a number of parts $k_1, k_2, k_3, \dots, k_n$ where each parts are added together and last carry is ignored. The hash function is defined as:

$$H(k) = k_1 + k_2 + k_3 + \dots + k_n$$

For example: Suppose we have 4 digit key 9128 and hash table consists 100, 2digit number.

Here, For $k= 9128$

We have , $H(k) = k_1 + k_2 + k_3 + \dots + k_n$

$$H(9128) = 91 + 28 = 119 = 19 \text{ (discard last carry1)}$$

For $k = 12345$

$$H(12345) = 12 + 34 + 5 = 51$$

Basic Operations

Following are basic primary operations of a Hash table which are following. **1. Insert Operation**

Whenever an element is to be inserted; Compute the hash code of the key passed and locate the index using that Hash code as index in the array. Use probing methods for empty location if an element is found at computed hash code.

2. Deletion Operation

Whenever an element is to be deleted; Compute the hash code of the key passed and locate the index using that Hash code as index in the array. Use probing methods to get element ahead if an element is not found at computed hash code. When found, store a dummy item there to keep performance of Hash table intact.

3. Search Operation

Whenever an element is to be searched; Compute the hash code of the key passed and locate the element using that Hash code as index in the array. Use probing methods to get element ahead if element not found at computed hash code.

Collision Resolution

In real practice an ideal excess to the table is rarely found where each key have unique address in a hash table. There may arises a situation when more than one different data item(key) may fill the same array index. In which we are trying to store a data but the place may be already occupied by the other data is called collision.

So it is needed to establish some methods of handling collision. Such a way both data can be store and can be retrieve. This process/method is known as collision resolution. **When two items hash to the same slot, we must have a systematic method for placing the second item in the hash table. This process is called collision resolution. If the hash function is perfect, collisions will never occur. However, since this is often not possible, collision resolution becomes a very important part of hashing.**

Data Structure And Algorithm Page 4

Unit-5: Hashing

1. A Separate-Chained Hash Table (Open Hashing):

Hashing with chaining is an application of linked list and gives an approach to collision resolution. In hashing with chaining, the hash table content link list of elements. The lists are referring to as chains and the technique is called chaining. Each linked list contains all the elements whose keys have the same index.

2. An Open-addressed Hash Table(Closed Hashing) :

In Open Addressing technique, the amount of space available for storing data is already fixed by declaring a fixed array for the hash table. There are two ways of open addressing method for collision resolution:

a. Linear Probing:

This is a simplest method for collision resolution. When collision occurs while we are inserting a new item into the table, we simply probe forward in the array one step at a time until we find an empty slot and we can store a new data.

b. Quadratic Probing

In Quadratic Probing method, if a collision occurs at position 'i', location $(i+1^2, i+2^2, i+3^2, \dots, i+j^2)$ where $j= 1,2,3, \dots, n$ are tested until an

empty slot is found.

Quadratic Probing is similar to linear probing. The difference is that if you were to try to insert into a space that is filled you would first check $1^2=1$ element away then $2^2=4$ elements away, then $3^2=9$ elements away then $4^2=16$ elements away and so on.

3. Double Hashing/ De-hashing

Double hashing uses the idea of applying a second hash function to the key when a collision occurs. The result of the second hash function will be the numbers of positions form the point of collision to insert.

There are a couple of requirements for the second function:

- It must never evaluate to 0
- Must make sure that all cells can be probed

A popular second hash function is: $\text{Hash}_2(\text{key}) = R - (\text{key} \% R)$ where R is a prime number that is smaller than the size of the table. We apply the second hash function to 'x' and probe at distance $h_2(x)$. Still collision occur then we apply $(h_1(x) + h_2(x)) \bmod m$, $(h_1(x) + 2h_2(x)) \bmod m$and so on to probe at distance with occur value until an empty slot wont found.

Load Factor

- The load factor of a hash table is the ratio of the number of keys in the table to the size of the hash table.
- The higher the load factor, the slower the retrieval.
- With open addressing, the load factor cannot exceed 1. With chaining, the load factor often exceeds 1.
- Note that 6 of the 11 slots are now occupied. This is referred to as the **load factor**, and is commonly denoted by $\lambda = \text{number of items} / \text{table size}$. For this example, $\lambda = 6/11 = 0.54$.

Data Structure And Algorithm Page 5

Unit-5: Hashing

Rehashing

Once the hash table gets too full, the running time for operations will start to take too long and may fail. To solve this problem, a table at least twice the size of the original will be built and the elements will be transferred to the new table.

The new size of the hash table:

- should also be prime
- will be used to calculate the new insertion spot (hence the name

rehashing) This method is applicable:

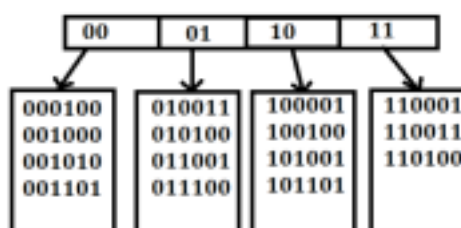
- Once the table becomes half full
- Once an insertion fails
- Once a specific load factor has been reached, where load factor is the ratio of the number of elements in the hash table to the table size

Extendible Hashing

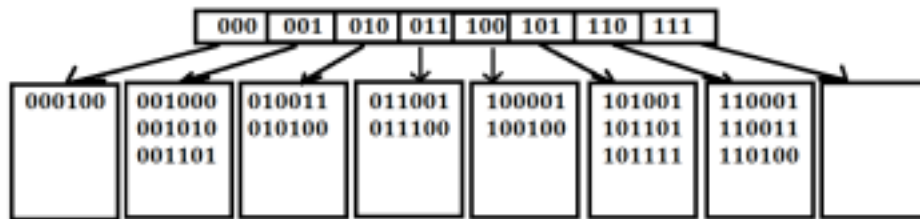
Extendable hashing is one form of dynamic hashing. Extendable hashing splits and combines buckets as database size changes. This imposes some performance overhead, but space efficiency is maintained. As reorganization is on one bucket at a time, overhead is acceptably low. **Extendible Hashing is a type of hash system which treats a hash as a bit string and used a trie(tree) for bucket lookup.** This technique is used when large number of elements is to be hashed with flexibility t increase in number of hash table.

The hash function is the first few bits of the key and tables refer to buckets. Table entries with the same initial bit may use the same bucket. If a bucket overflows it splits. Splitting of table cause the referring table to be doubled in size. Similarly, deletion of element and table may cause the referring table to be halved.

Let us consider an example with table size $M=4$.



Now, Let us suppose we want to insert 101111 which could go to the third leaf but the third leaf is already full. Thus we split the bucket as:



➤ Advantages

- Extendable hashing provides performance that does not degrade as the file grows.
- Minimal space overhead - no buckets need be reserved for future use. Bucket address table only contains one pointer for each hash value of current prefix length.

➤ Disadvantages

- Extra level of indirection in the bucket address table
- Added complexity

A. Consider the company with employee. Each employee is assigned a unique 4-digit number. Suppose hash table consider 100, 2-digit number. Apply the hash function to given keys with all three methods. Keys: 3205, 7148, 2345

1. Division Method :

Let $m=97$ (nearest prime no. of $n=100$)

We know that,

$$h(k) = k \bmod m$$

$$\text{For } k= 3205, h(3205) = 3205 \bmod 97 = 04$$

$$\text{For } k= 7148, h(7148) = 7148 \bmod 97 = 67$$

$$\text{For } k= 2345, h(2345) = 2345 \bmod 97 = 17$$

2. Mid- Square Method :

Key value square address $H(k)=L$

$$3205 \ 10272025 \ 72 \ h(3205) = 72$$

$$7148 \ 51093904 \ 93 \ h(7148) = 93$$

$$2345 \ 05499025 \ 99 \ h(2345) = 99$$

3. Folding Method :

We have , $H(k) = k_1 + k_2 + k_3 \dots + k_n$

Here,

For $k= 3205$,

$$H(3205)= 32+05 = 37$$

For $k= 7148$,

$$H(7148) = 71+48 = 119 = 19 \text{ (discard last carry 1)}$$

For $k= 2345$,

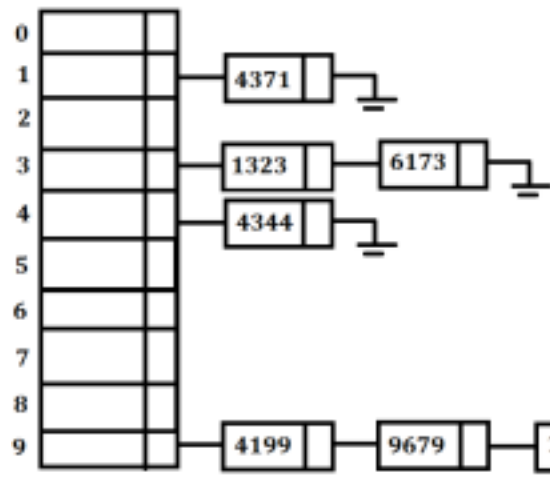
$$H(2345) = 23+45 = 68$$

- B. For the given elements: {4371,1323,6173,4199,4344,9679,1989} and a hash function $h(x)=x \bmod 10$. Show the following:**
- Separate Chaining hash table**
 - Open addressing hash table**
 - Using linear probing**
 - Using Quadratic Probing**
 - Open addressing hash table with second hash function: $h_2(x) = 7-x \bmod 7$.**
- Solution:**

Here, hash function $h(x) = x \bmod 10$
 For key 4371 , $h(4371) = 4371 \bmod 10 = 1$
 For key 1323, $h(1323) = 1323 \bmod 10 = 3$
 For key 6173, $h(6173) = 6173 \bmod 10 = 3$
 For key 4199, $h(4199) = 4199 \bmod 10 = 9$
 For key 4344, $h(4344) = 4344 \bmod 10 = 4$
 For key 9679, $h(9679) = 9679 \bmod 10 = 9$
 For key 1989, $h(1989) = 1989 \bmod 10 = 9$

- a. Separate Chaining:

0	9679
1	4371
2	1989
3	1323
4	6173
5	4344
6	
7	
8	
9	4199



b. i. Open addressing hash

table with linear probing: Here, when collision occurs we search for next free slot and insert key on that free slot.

0	9679
1	4371
2	
3	1323
4	6173
5	4344
6	
7	
8	1989
9	4199

ii. Open addressing hash table with quadratic probing: Here , When a collision occurs at any position 'i' , location $(i+1^2, i+2^2, i+3^2, \dots, i+j^2)$ where $j= 1,2,3,\dots,n$ are tested until an empty slot is found.

0	1989
1	4371
2	9679
3	1323
4	6173
5	
6	
7	4344
8	
9	4199

c. Here,

Second Hash Function $h_2(x)$ for all collide key:

For key 6173, $h_2(6173) = 7-6173 \bmod 7 = 7-6 = 1$

For key 4344, $h_2(4344) = 7-4344 \bmod 7 = 7-4 = 3$

For key 9679, $h_2(9679) = 7-9679 \bmod 7 = 7-5 = 2$, collision occur.

$$(h_1(x) + 2h_2(x)) \bmod 10 = (9+4) \bmod 10 = 3$$

For key 1989, $h_2(1989) = 7-1989 \bmod 7 = 7-1 = 6$, collision occur.

$$(h_1(x) + 2h_2(x)) \bmod 7 = (9+12) \bmod 10 = 1$$

Rehashing:

There are chances of insertion failure when hash table is full, so the solution for this particular case is to create a new hash table with the double size of previous hash table. Here, we will use new hash function and we will insert all the elements of the previous hash table. So, we will scan the elements of previous hash table one by one & calculate the hash key with new hash function & insert them into new hash table.

Consider the table size 11 & elements are 7, 18, 43, 10, 36, 25..... Soln:-

Size = 11

Applying linear probing, we get :

$$h(7) = 7 \bmod 11 = 7$$

$$h(18) = 18 \bmod 11 = 7$$

$$h(43) = 43 \bmod 11 = 10$$

$$h(10) = 10 \bmod 11 = 10$$

$$h(36) = 36 \bmod 11 = 3$$

$$h(25) = 25 \bmod 11 = 3$$

Now, if we want to insert six more element then size will not be sufficient. In order to fit all the elements or key with in a table we take new table of size more than double with prime number.

Thus total size is 23.

Then, we get,

$$h(7) = 7 \bmod 23 = 7$$

$$h(18) = 18 \bmod 23 = 18$$

$$h(43) = 43 \bmod 23 = 20$$

$$h(10) = 10 \bmod 23 = 10$$

$$h(36) = 36 \bmod 23 = 13$$

$$h(25) = 25 \bmod 23 = 2$$

Disadvantage:- This method is more expression.

0	10
1	
2	
3	36
4	25
5	
6	
7	7
8	18
9	
10	43

0	
1	
2	25
3	
4	
5	
6	
7	
8	
9	
10	10
11	
12	
13	36
14	
15	
16	
17	
18	18
19	
20	43
21	
22	

